p0847R0
Deducing this

Published Proposal, 12 February 2018

This version:
http://wg21.link/P0847R0

Authors:
Gašper Ažman (gasper dot azman at gmail dot com)
Simon Brand (simon at codeplay dot com)
Ben Deane (ben at elbeno dot com)
Barry Revzin (barry dot revzin at gmail dot com)

Audience:
EWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

We propose a new mechanism for specifying or deducing the value category of an instance of a class. In other words, a way to tell from within a member function whether the object it's invoked on is an lvalue or an rvalue, and whether it is const or volatile.

Table of Contents

§ 1. Motivation

In C++03, member functions could have *cv*-qualifications, so it was possible to have scenarios where a particular class would want both a `const` and non-`const` overload of a particular member (Of course it was possible to also want `volatile` overloads, but those are less common). In these cases, both overloads do the same thing - the only difference is in the types accessed and used. This was handled by either simply duplicating the function, adjusting types and qualifications as necessary, or having one delegate to the other. An example of the latter can be found in Scott Meyers' "Effective C++", Item 3:

```
class TextBlock {
public:
  const char& operator[](std::size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](std::size_t position) {
    return const_cast<char&>(
      static_cast<const TextBlock&>(this)[position]
    );
  }
  // ...
};
```

Arguably, neither the duplication or the delegation via `const_cast` are great solutions, but they work.

In C++11, member functions acquired a new axis to specialize on: ref-qualifiers. Now, instead of potentially needing two overloads of a single member function, we might need four: `&`, `const&`, `&&`, or `const&&`. We have three approaches to deal with this: we implement the same member four times, we can have three of the overloads delegate to the fourth, or we can have all four delegate to a helper, private static member function. One example might be the overload set for `optional<T>::value()`. The way to implement it would be something like:

Quadruplication	Delegation to 4th	Delegation to help

```
template <typename T>
class optional {
    // ...
    constexpr T& value() & {
        if (has_value()) {
            return this->m_value;
        }
        throw bad_optional_access();
    }

    constexpr const T& value() const& {
        if (has_value()) {
            return this->m_value;
        }
        throw bad_optional_access();
    }

    constexpr T&& value() && {
        if (has_value()) {
            return std::move(this->m_value);
        }
        throw bad_optional_access();
    }

    constexpr const T&&
    value() const&& {
        if (has_value()) {
            return std::move(this->m_value);
        }
        throw bad_optional_access();
    }
    // ...
};
```

```
template <typename T>
class optional {
    // ...
    constexpr T& value() & {
        return const_cast<T&>(
            static_cast<optional const&>(
                *this).value());
    }

    constexpr const T& value() const& {
        if (has_value()) {
            return this->m_value;
        }
        throw bad_optional_access();
    }

    constexpr T&& value() && {
        return const_cast<T&&>(
            static_cast<optional const&>(
                *this).value());
    }

    constexpr const T&&
    value() const&& {
        return static_cast<const T&&>(
            value());
    }
    // ...
};
```

```
template <typen
class optional
    // ...
    constexpr T
        return
    }

    constexpr c
        return
    }

    constexpr T
        return
    }

    constexpr c
    value() con
        return
    }

private:
    template <t
    static decl
    value_impl(
        if (!op
            thr
        }
        return
    }


    // ...
};
```

It's not like this is a complicated function. Far from. But more or less repeating the same code four times, or artificial delegation to avoid doing so, is the kind of thing that begs for a rewrite. Except we can't really. We *have* to implement it this way. It seems like we should be able to abstract away the qualifiers. And we can... sort of. As a non-member function, we simply don't have this problem:

```
template <typename T>
class optional {
    // ...
    template <typename Opt>
    friend decltype(auto) value(Opt&& o) {
        if (o.has_value()) {
            return std::forward<Opt>(o).m_value;
        }
        throw bad_optional_access();
    }
    // ...
};
```

This is great - it's just one function, that handles all four cases for us. Except it's a non-member function, not a member function. Different semantics, different syntax, doesn't help.

There are many, many cases in code-bases where we need two or four overloads of the same member function for different `const` - or ref-qualifiers. More than that, there are likely many cases that a class should have four overloads of a particular member function, but doesn't simply due to laziness by the developer. We think that there are sufficiently many such cases that they merit a better solution than simply: write it, then write it again, then write it two more times.

§ 2. Proposal

We propose a new way of declaring a member function that will allow for deducing the type and value category of the class instance parameter, while still being invocable as a member function. We introduce a new kind of parameter that can be provided as the first parameter to any member function: an explicit object parameter. The purpose of this parameter is to bind to the implicit object, allowing it to be deduced.

An explicit object parameter shall be of the form `T [const] [volatile] [&|&&] this <identifier>`, where `T` follows the normal rules of type names (e.g. for generic lambdas, it can be `auto`), except that it cannot be a pointer type. Member functions with explicit object parameters cannot be `static` or have *cv*- or *ref*-qualifiers.

The explicit object parameter can only be the first parameter of a function. It cannot be used in constructors or destructors.

This is a strict extension to the language; all existing syntax remains valid.

For the purposes of this proposal, we assume the existence of two new library functions: a metafunction named `like` which applies the *cv* and *ref* qualifiers from its first type argument onto its second (such that `like_t<int&, double>` is `double&`, `like_t<X const&&, Y>` is `Y const&&`, etc.), and a function template named `forward_like` which allows you to forward based on the value category of an unrelated type (`forward_like<T>(u)` is short-hand for `forward<like_t<T, U>>(u)`). Sample implementations of both can be seen here.

With this extension, the example from above can be written like so:

```
template <typename T>
class optional {
    // ...
    template <typename Self>
    decltype(auto) value(Self&& this) {
        if (o.has_value()) {
            return forward_like<Self>(*this).m_value;
        }
        throw bad_optional_access();
    }
    // ...
};
```

We believe that the ability to write *cv-ref qualifier*-aware member functions without duplication will improve code maintainability, decrease the likelihood of bugs, and allow users to write fast, correct code more easily.

What follows is a description of how explicit object parameters affect all the important language constructs: name lookup, type deduction, overload resolution, and so forth.

§ 2.1. Name lookup: candidate functions

Today, when either invoking a named function or an operator (including the call operator) on an object of class type, name lookup will include both static and non-static member functions found by regular class lookup. Non-static member functions are treated as if there were an implicit object parameter whose type is an lvalue or rvalue reference to *cv* `X` (where the reference and *cv* qualifiers are determined based on the function's qualifiers) which binds to the object on which the function was invoked. For static member functions, the implicit object parameter is effectively discarded, so they will not be mentioned further.

For non-static member functions with the new **explicit** object parameter, lookup will work the same way as other member functions today, except rather than implicitly determining the type of the object parameter based on the *cv*- and *ref*-qualifiers of the member function, these are set by the parameter itself. The following examples illustrate this concept. Note that the explicit object parameter does not need to be named:

C++17	With Explicit Object

```cpp
struct X {
    // implicit object has type X&
    void foo();

    // implicit object has type X const&
    void foo() const;

    // implicit object has type X&&
    void bar() &&;

    /* ex_baz has no C++17 equivalent */
};
```

```cpp
struct X {
    // explicit object, named self, has type X&
    void ex_foo(X& this self);

    // explicit object, named self, has type X const&
    void ex_foo(X const& this self);

    // explicit object, unnamed, has type X&&
    void ex_bar(X&& this);

    // explicit object, unnamed, has type X
    // copied or moved from original object
    void ex_baz(X this);
};
```

The overload resolution rules for this new set of candidate functions remains unchanged - we're simply being explicit rather than implicit about the object parameter. Given a call to `x.ex_foo()`, overload resolution would select the first `ex_foo()` overload if `x` isn't `const` and the second if it is.

Since the first parameter in member functions that have an explicit object parameter binds to the class object on which the member function is being invoked, the first provided argument is used to initialize the second function parameter, if any:

```cpp
struct C {
    int get(C const& this self, int i);
};

C c;
c.get(4); // self is a C const&, initialized to c
          // i is initialized with 4
```

§ 2.2. Type deduction

One of the main motivations of this proposal is to deduce the *cv*-qualifiers and value category of the class object, so the explicit object parameter needs to be deducible. We do not propose any change in the template deduction rules for member functions with explicit object parameters - it will just be deduced from the class object the same way as any other function parameter:

```cpp
struct X {
    template <typename Self>
    void foo(Self&& this self, int i);
};

X x;
x.foo(4); // Self deduces as X&
std::move(x).foo(2); // Self deduces as X
```

Since deduction rules do not change, and the explicit object parameter is deduced from the object the function is called on, this has the interesting effect of possibly deducing derived types, which can best be illustrated by the following example:

```
struct B {
    int i = 0;

    template <typename Self>
    auto&& get(Self&& this self) {
        // NB: specifically self.i, not just i or this->i
        return self.i;
    }
};

struct D : B {
    // shadows B::i
    double i = 3.14;
};

B b{};
B const cb{};
D d{};

b.foo();            // #1
cb.foo();           // #2
d.foo();            // #3
std::move(d).foo(); // #4
```

The proposed behavior of these calls is:

1. `Self` is deduced as `B&`, this call returns an `int&` to `B::i`
2. `Self` is deduced as `B const&`, this calls returns an `int const&` to `B::i`
3. `Self` is deduced as `D&`, this call returns a `double&` to `D::i`
4. `Self` is deduced as `D`, this call returns a `double&&` to `D::i`

When we deduce the object parameter, we don't just deduce the *cv*- and *ref*-qualifiers. We may also get a derived type. This follows from the normal template deduction rules. In `#3`, for instance, the object parameter is an lvalue of type `D`, so `Self` deduces as `D&`.

§ 2.3. Name lookup: within member functions with explicit object parameters

So far, we've only considered how member functions with explicit object parameters get found with name lookup and how they deduce that parameter. Now let's move on to how the bodies of these functions actually behave. Consider a slightly expanded version of the previous example:

```
struct B {
    int i = 0;

    template <typename Self>
    auto&& f1(Self&& this self) { return i; }

    template <typename Self>
    auto&& f2(Self&& this self) { return this->i; }

    template <typename Self>
    auto&& f3(Self&& this self) { return std::forward<Self>(*this).i; }

    template <typename Self>
    auto&& f4(Self&& this self) { return std::forward<Self>(self).i; }
};

struct D : B {
    double i = 3.14;
};
```

Consider invoking each of these functions with an lvalue of type `D`. We have no interest in changing template deduction rules, which means that `Self` in each case will be `D&`. The design space consists of the following alternatives:

1. `this` is a `B*` (non-`const` because the explicit object parameter is non-`const`), unqualified lookup looks in `B`'s scope first. This means that `f1` returns an `int&` (to `B::i`) and `f2` returns an `int&`. `f3` is ill-formed, because `std::forward<D&>` takes a `D&` and we're passing in a `B&`, which is not a conversion that can be done implicitly. `f4` returns a `double&` to `D::i`, because while `*this` is a `B`, `self` actually is the object parameter, hence is a `D`.
2. `this` is a `D*` (non-`const`), but unqualified lookup looks in `B`'s scope. This means that `f1` returns an `int&`, but `f2` returns a `double&`. `f3` and `f4` both return `double&`.
3. `this` is a `D*` and unqualified lookup looks in `D`'s scope too. All four functions return a `double&`.
4. While the previous three options are variations on these functions behaving as non-static member functions, we could instead also consider these functions to behave as either static member functions or non-member `friend` functions. In this case, any member access would require direct access from the explicit object parameter, so `f1`, `f2`, and `f3` are all ill-formed. `f4` returns a `double&`.

In our view, options 2 and 3 are too likely to be lead to programmer errors, even by experts. It would be very surprising if `f1` didn't return `B::i`, or if `f1` and `f2` behaved differently. Option 4 is interesting, but would be more verbose without clear benefit.

We favor option 1, since `f1`, `f2` and `f4` all end up having behavior that is reasonable given the other rules for name lookup we already have. `f2` and `f4` returning different things might appear strange at first glance, but `f2` references `this->i` and `f4` references `self.i` - programmers would have to be aware that these may be different. Where this option could go wrong is `f3`, which looks like it yields an lvalue or rvalue reference to `B::i` but is actually casting `*this` to a derived object. But since this is ill-formed, this is a *compile* error rather than a runtime bug. Hence, not only do we get typically expected behavior, but we're also protected from a potential source of error by the compiler.

The proper way to implement forwarding is to use one of the two hypothetical library functions mentioned earlier:

```
struct B {
    template < typename Self>
    auto&& f3(Self&& this self) {
        return std::forward<Self>(*this).i;            // ok if Self and *this are the same
                                                       // compile other if Self is a derive

        return std::forward<like_t<Self, B>>(*this).i; // always ok
        return std::forward_like<Self>(*this).i;       // always ok
    }
};
```

The rule that we're establishing is that `this` remains a pointer to the class type of the member function it is used in, and unqualified lookup looks in class scope as usual. Additionally, `this` is either a pointer to the explicit object parameter or, in the case of deducing a derived type, one of its base class subobjects.

§ 2.4. By-value explicit object parameters

We think they are a logical extension of the mechanism, and would go a long way towards making member functions as powerful as inline friend functions, with the only difference being the call syntax. One implication of this is that the explicit object parameter would be move-constructed in cases where the object is an rvalue (or constructed in place for prvalues), allowing you to treat chained builder member functions that return a new object uniformly without having to resort to templates.

We continue to follow the rule established in the previous section: `this` is a pointer to the explicit object parameter or one of its base class subobjects. In the example below, `this == &self` and all accesses to `s` could also be rewritten as `this->s` or `self.s`:

```
class string_builder {
  std::string s;

  operator std::string(string_builder this self) {
    return std::move(s);
  }

  string_builder operator*(string_builder this self, int n) {
    assert(n > 0);

    s.reserve(s.size() * n);
    auto const size = s.size();
    for (auto i = 0; i < n; ++i) {
      s.append(s, 0, size);
    }
    return self;
  }

  string_builder bop(string_builder this self) {
    s.append("bop");
    return self;
  }
};

// this is optimally efficient as far as allocations go
std::string const x = (string_builder{"asdf"} * 5).bop().bop();
```

In this example, `x` would hold the value `"asdfasdfasdfasdfasdfbopbop"`.

Of course, implementing this example with templated explicit object parameter member functions would have been slightly more efficient due to also saving on move constructions, but the by-value usage makes for simpler code.

§ 2.5. Writing the function pointer types for such functions

Currently, we write member function pointers like so:

```
struct Y {
  int f(int a, int b) const &;
};
static_assert(std::is_same_v<decltype(&Y::f), int (Y::*)(int, int) const &>);
```

All the member functions that take references already have a function pointer syntax - they are just alternate ways of writing functions we can already write.

The only one that does not have such a syntax is the pass-by-value method, all others have pre-existing signatures that do just fine.

We are asking for suggestions for syntax for these function pointers. We give our first pass here:

```
struct Z {
  // same as int f(int a, int b) const&;
  int f(Z const& this, int a, int b);

  int g(Z this, int a, int b);
};

// f is still the same as Y::f
static_assert(std::is_same_v<decltype(&Z::f), int (Z::*)(int, int) const &>);
// but would this alternate syntax make any sense?
static_assert(std::is_same_v<decltype(&Z::f), int (*)(Z::const&, int, int)>);
// It allows us to specify the syntax for Z as a pass-by-value member function
static_assert(std::is_same_v<decltype(&Z::g), int (*)(Z::, int, int)>);
```

Such an approach unifies, to a degree, the member functions and the rest of the function type spaces, since it communicates not only that the first parameter is special, but also its type and calling convention.

§ 2.6. Teachability Implications

Using `auto&& this self` follows existing patterns for dealing with forwarding references.

Explicitly naming the object as the `this`-designated first parameter fits with many programmers' mental model of the `this` pointer being the first parameter to member functions "under the hood" and is comparable to usage in other languages, e.g. Python and Rust.

This also makes the definition of "`const` member function" more obvious, meaning it can more easily be taught to students.

It also works as a more obvious way to teach how `std::bind` and `std::function` work with a member function pointer by making the pointer explicit.

§ 2.7. ABI implications for `std::function` and related

If references and pointers do not have the same representation for member functions, this effectively says "for the purposes of the `this`-designated first parameter, they do."

This matters because code written in the "`this` is a pointer" syntax with the `this->` notation needs to be assembly-identical to code written with the `self.` notation; the two are just different ways to implement a function with the same signature.

§ 2.8. `virtual` and `this` as value

Virtual member functions are always dispatched based on the type of the object the dot -- or arrow, in case of pointer -- operator is being used on. Once the member function is located, the parameter `this` is constructed with the appropriate move or copy constructor and passed as the `this` parameter, which might incur slicing.

Effectively, there is no change from current behavior -- only a slight addition of a new overload that behaves the way a user would expect.

Virtual member function templates are not allowed in the language today, and this paper does not propose a change from the current behavior for member functions with explicit object parameters either.

§ 2.9. Can `static` member functions have a `this` parameter?

No. Static member functions currently do not have an implicit `this` parameter, and therefore have no reason to have an explicit one.

§ 2.10. Interplays with capturing `[this]` and `[*this]` in lambdas

In the function parameter list, `this` just designates the explicit object parameter. It does not, in any way, change the meaning of `this` in the body of the lambda.

C++17

```
struct X {
    int x, y;

    auto getter() const
    {
        return [*this](){
            return x       // refers to X::x
                + this->y; // refers to X::y
        };
    }
};
```

Proposed

```
struct X {
    int x, y;

    auto getter() const
    {
        return [*this](auto const& this /* self */){
            return x       // still refers to X::x
                + this->y; // still refers to X::y
        };
    }
};
```

If other language features play with what `this` means, they are completely orthogonal and do not have interplays with this proposal. However, it should be obvious that developers have great potential for introducing hard-to-read code if they are at all changing the meaning of `this` in function bodies, especially in conjunction with this proposal.

§ 2.11. Translating code to use explicit object parameters

The most common qualifier overload sets for member functions are:

1. `const` and non-`const`
2. `&`, `const&`, `&&`, and `const&&`
3. `const&` and `&&`

Some examples:

1

```
struct foo {
  void bar();
  void bar() const;
};
```

2

```
struct foo {
  void bar() &;
  void bar() const&;
  void bar() &&;
  void bar() const&&;
};
```

3

```
struct foo {
  void bar() const&;
  void bar() &&;
};
```

All three of these can be handled by a single perfect-forwarding overload, like this:

```
struct foo {
    template <class Self>
    void bar (Self&& this self);
};
```

This overload is callable for all `const`- and ref-qualified object parameters, just like the above examples. It is also callable for `volatile`-qualified objects, so the code is not entirely equivalent; however, the `volatile` versions are unlikely to be valid and more likely to be simply left out for the sake of brevity. The only major difference is in the third case, where non-`const` lvalue arguments would be non-`const` inside the function body, and `const` rvalue arguments would be `const&&` instead of `const&`. Again, this is unlikely to cause correctness issues unless `this` has other member functions called on it which do semantically different things depending on the cv-qualification of `this`.

§ 2.12. Alternative syntax

Instead of qualifying the parameter with `this`, to mark it as the parameter for deducing `this`, we could let the first parameter be *named* `this` instead:

```
template <typename T>
struct X {
    T value;

    // as proposed
    template <typename Self>
    decltype(auto) foo(Self&& this self) {
        return std::forward<Self>(self).value;
    }

    // alternative
    template <typename This>
    decltype(auto) foo(This&& this) {
        return std::forward<This>(this).value;
    }
```

In the example above, `this` is no longer a pointer. Since `this` can never be a null pointer in valid code in the first place, this is an advantage, but may add confusion to the language. It also leaves us at a loss as to how one can address the derived object and the base object, as outlined in the previous section.

Rather than naming the first parameter `this`, we can also consider introducing a dummy template parameter where the qualifications normally reside. This syntax is also ill-formed today, and is purely a language extension:

```
template <typename T>
struct X {
    T value;

    // as proposed
    template <typename Self>
    decltype(auto) foo(Self&& this self) {
        return std::forward<Self>(self).value;
    }

    // alternative
    template <typename This>
    decltype(auto) foo() This&& {
        return std::forward<This>(*this).value;
    }

    // another alternative
    decltype(auto) foo() auto&& {
        return std::forward<decltype(*this)>(*this).value;
    }
};
```

This has the benefit of not muddying the parameter list, and it keeps the qualifier deduction off to the side where qualifiers usually live.

§ 2.13. Unified Function Call Syntax

The proposed use of `this` as the first parameter seems as if these members functions really had better be written as non-members to begin with, and might suggest that the solution to this problem is really unified function call syntax (UFCS).

However, several of the motivating examples presented in this proposal are implementations of operators that cannot be implemented as non-members (`()`, `[]`, `->`, and unary `*`). UFCS alone would be insufficient.

Additionally, the member function overload sets in all of these examples exist as *member* functions, not free functions, today. Having to take a member function overload set and rewrite it as a non-member (likely `friend` ed) free function template is very much changing the intended design of a class to overcome a language hurdle. This proposal contends that it would be more in keeping with programmer intent to allow member functions to stay member functions.

§ 3. Real-World Examples

§ 3.1. Deduplicating Code

This proposal can de-duplicate and de-quadruplicate a large amount of code. In each case, the single function is only slightly more complex than the initial two or four, which makes for a huge win. What follows are a few examples of how repeated code can be reduced.

The particular implementation of optional is Simon's, and can be viewed on GitHub, and this example includes some functions that are proposed in P0798, with minor changes to better suit this format:

C++17

```
class TextBlock {
public:
  const char&
  operator[](std::size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](std::size_t position) {
    return const_cast<char&>(
      static_cast<const TextBlock&>
        (this)[position]
    );
  }
  // ...
};
```

This proposal

```
class TextBlock {
public:
  template <typename Self>
  auto& operator[](Self&& this,
                   std::size_t position) {
    // ...
    return text[position];
  }
  // ...
};
```

C++17

```
template <typename T>
class optional {
  // ...
  constexpr T* operator->() {
    return std::addressof(this->m_value);
  }

  constexpr const T*
  operator->() const {
    return std::addressof(this->m_value);
  }
  // ...
};
```

This proposal

```
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr auto operator->(Self&& this) {
    return std::addressof(this->m_value);
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  constexpr T& operator*() & {
    return this->m_value;
  }

  constexpr const T& operator*() const& {
    return this->m_value;
  }

  constexpr T&& operator*() && {
    return std::move(this->m_value);
  }

  constexpr const T&&
  operator*() const&& {
    return std::move(this->m_value);
  }

  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr const T& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return std::move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr const T&& value() const&& {
    if (has_value()) {
      return std::move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr like_t<Self, T>&& operator*(Self&& this) {
    return forward_like<Self>(*this).m_value;
  }

  template <typename Self>
  constexpr like_t<Self, T>&& value(Self&& this) {
    if (has_value()) {
      return forward_like<Self>(*this).m_value;
    }
    throw bad_optional_access();
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  template <typename F>
  constexpr auto and_then(F&& f) & {
    using result =
      invoke_result_t<F, T&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) && {
    using result =
      invoke_result_t<F, T&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 std::move(**this))
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const& {
    using result =
      invoke_result_t<F, const T&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const&& {
    using result =
      invoke_result_t<F, const T&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 std::move(**this))
        : nullopt;
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  template <typename Self, typename F>
  constexpr auto
  and_then(Self&& this, F&& f) {
    using val = decltype((
        forward_like<Self>(*this).m_value));
    using result = invoke_result_t<F, val>;

    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 forward_like<Self>
                   (self).m_value)
        : nullopt;
  }
  // ...
};
```

Keep in mind that there are a few more functions in P0798 that have this lead to this explosion of overloads, so the code difference and clarity is dramatic.

For those that dislike returning auto in these cases, it is very easy to write a metafunction that matches the appropriate qualifiers from a type. Certainly simpler than copying and pasting code and hoping that the minor changes were made correctly in every case.

§ 3.2. Recursive Lambdas

This proposal also allows for an alternative solution to implementing a recursive lambda, since now we open up the possibility of allowing a lambda to reference itself:

```
// as proposed in [P0839]
auto fib = [] self (int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};

// this proposal
auto fib = [](auto& this self, int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};
```

This simply works following the established rules. The call operator of the closure object can have an explicit object parameter too, so `self` in this example *is* the closure object.

If the lambda would otherwise decay to a function pointer, `&self` shall have the value of that function pointer.

§ 3.2.1. Expressions allowed for `self` in lambdas

```
  self(...);      // call with appropriate signature
  decltype(self); // evaluates to the type of the lambda with the appropriate
                  // cv-ref qualifiers
  &self;          // the address of either the closure object or function pointer
  std::move(self) // you're allowed to move yourself into an algorithm...
  /* ... and all other things you're allowed to do with the lambda itself. */
```

Within lambda expressions, the `this` parameter still does not allow one to refer to the members of the closure object, which has no defined storage or layout, nor do its members have names. Instead it allows one to deduce the value category of the lambda and access its members -- including various call operators -- in the way appropriate for the value category.

§ 3.2.2. Deducing derived objects for generic lambdas

When we deduce the object parameter, it doesn't have to be the same type as the class - it could be a derived type. This isn't typically relevant for lambdas, which would just be used as is, but it does provide an interesting opportunity when it comes to visitation. One tool to make it easier to invoke `std::visit()` on a `std::variant` is to write an overload utility which ends up creating a new type that inherits from all of the provided types. Consider a scenario where we have a `Tree` type which is a `variant<Leaf, Node>`, where a `Node` contains two sub-`Tree`s. We could count the number of leaves in a `Tree` as follows:

```
int num_leaves(Tree const& tree) {
    return std::visit(overload(
        [](Leaf const&) { return 1; },
        [](auto const& this self, Node const& n) {
            return std::visit(self, n.left) + std::visit(self, n.right);
        }
    ), tree);
}
```

In this example, `self` doesn't deduce as that inner lambda, it deduces at the overload object, so this works.

§ 3.3. CRTP, without the C, R, or even T

Today, a common design pattern is the Curiously Recurring Template Pattern. This implies passing the derived type as a template parameter to a base class template, as a way of achieving static polymorphism. If we wanted to just outsource implementing postfix incrementing to a base, we could use CRTP for that. But with explicit object parameters that deduce to the derived objects already, we don't need any curious recurrence. We can just use standard inheritance and let deduction just do its thing. The base class doesn't even need to be a template:

C++17

```
template <typename Derived>
struct add_postfix_increment {
    Derived operator++(int) {
        auto& self = static_cast<Derived&>(*this);

        Derived tmp(self);
        ++self;
        return tmp;
    }
};

struct some_type : add_postfix_increment<some_type> {
    some_type& operator++() { ... }
};
```

Proposed

```
struct add_postfix_increment {
    template <typename Self>
    Self operator++(Self& this self, int) {
        Self tmp(self);
        ++self;
        return tmp;
    }
};



struct some_type : add_postfix_increment {
    some_type& operator++() { ... }
};
```

The example at right isn't much shorter, but it is certainly simpler.

§ 3.4. SFINAE-friendly callables

A seemingly unrelated problem to the question of code quadruplication is that of writing these numerous overloads for function wrappers, as demonstrated in P0826. Consider what happens if we implement `std::not_fn()`, as currently specified:

```
template <typename F>
class call_wrapper {
    F f;
public:
    // ...
    template <typename... Args>
    auto operator()(Args&&... ) &
        -> decltype(!declval<invoke_result_t<F&, Args...>>());

    template <typename... Args>
    auto operator()(Args&&... ) const&
        -> decltype(!declval<invoke_result_t<const F&, Args...>>());

    // ... same for && and const && ...
};

template <typename F>
auto not_fn(F&& f) {
    return call_wrapper<std::decay_t<F>>{std::forward<F>(f)};
}
```

As described in the paper, this implementation has two pathological cases: one in which the callable is SFINAE-unfriendly (which would cause a call to be ill-formed, when it could otherwise work), and one in which overload is deleted (which would cause a call to fallback to a different overload, when it should fail):

```
struct unfriendly {
    template <typename T>
    auto operator()(T v) {
        static_assert(std::is_same_v<T, int>);
        return v;
    }

    template <typename T>
    auto operator()(T v) const {
        static_assert(std::is_same_v<T, double>);
        return v;
    }
};

struct fun {
    template <typename... Args>
    void operator()(Args&&...) = delete;

    template <typename... Args>
    bool operator()(Args&&...) const { return true; }
};

std::not_fn(unfriendly{})(1); // static assert!
                              // even though the non-const overload is viable and would be
                              // match, during overload resolution, both overloads of unfri
                              // to be instantiated - and the second one is a hard compile

std::not_fn(fun{})();         // ok!? Returns false
                              // even though we want the non-const overload to be deleted,
                              // overload of the call_wrapper ends up being viable - and th
                              // candidate.
```

Gracefully handling SFINAE-unfriendly callables is **not solvable** in C++ today. Preventing fallback can be solved by the addition of yet another four overloads, so that each of the four *cv*/ref-qualifiers leads to a pair of overloads: one enabled and one `deleted`.

This proposal solves both problems by simply allowing `this` to be deduced. The following is a complete implementation of `std::not_fn`:

```
template <typename F>
struct call_wrapper {
    F f;

    template <typename Self, typename... Args>
    auto operator()(Self&& this, Args&&... args)
        -> decltype(!invoke(forward_like<Self>(f), forward<Args>(args)...))
    {
        return !invoke(forward_like<Self>(f), forward<Args>(args)...);
    }
};

template <typename F>
auto not_fn(F&& f) {
    return call_wrapper<decay_t<F>>{forward<F>(f)};
}

not_fn(unfriendly{})(1); // ok
not_fn(fun{})();         // error
```

Here, there is only one overload with everything deduced together. The first example now works correctly. `Self` gets deduced as `call_wrapper<unfriendly>`, and the one `operator()` will only consider `unfriendly`'s

non-`const` call operator. The `const` one is simply never considered, so does not have an opportunity to cause problems. The call works.

The second example now fails correctly. Previously, we had four candidates: the two non-`const` ones were removed from the overload set due to `fun`'s non-`const` call operator being `delete`d, and the two `const` ones which were viable. But now, we only have one candidate. `Self` gets deduced as `call_wrapper<fun>`, which requires `fun`'s non-`const` call operator to be well-formed. Since it is not, the call is an error. There is no opportunity for fallback since there is only one overload ever considered.

As a result, this singular overload then has precisely the desired behavior: working, for `unfriendly`, and not working, for `fun`.

§ 4. Acknowledgements

The authors would like to thank:

- Jonathan Wakely, for bringing us all together by pointing out we were writing the same paper, twice
- Graham Heynes, Andrew Bennieston, Jeff Snyder for early feedback regarding the meaning of `this` inside function bodies
- Chandler Carruth, Amy Worthington, Jackie Chen, Vittorio Romeo, Tristan Brindle, Agustín Bergé, Louis Dionne, and Michael Park for early feedback
- Guilherme Hartmann for his guidance with the implementation